

Python 高阶语言



学习目标：

1. 掌握正则表达式的基本语法，能够使用 `re` 模块进行字符串匹配、查找、替换和提取。
2. 理解 Python 的命名空间与作用域规则（LEGB），能够正确使用 `global` 和 `nonlocal`。
3. 理解迭代器与可迭代对象的区别，能够自定义迭代器。
4. 理解生成器的原理，能够使用 `yield` 编写生成器函数和生成器表达式。
5. 掌握生成器的 `send()`、`close()` 方法及其应用场景。
6. 掌握装饰器的编写方法，能够实现日志、计时、缓存、重试等常见装饰器。
7. 理解带参数装饰器和多个装饰器叠加的执行顺序。

第十八章：正则表达式（re模块）—— 文本处理的瑞士军刀

1. 为什么需要正则表达式？

定义：正则表达式（Regular Expression，简称 regex）是一种用特定字符组合来描述字符串模式的工具。它能够快速检查一个字符串是否与某种模式匹配、提取符合规则的部分、替换特定内容、切割字符串等。

生活类比：想象你在一个巨大的文档中查找所有电话号码。如果手动查找，你会搜索“可能是手机号的连续11位数字”。正则表达式就是把这个“可能是”的规则写出来，让计算机自动帮你找。

没有正则表达式之前：你需要写很多行代码来判断每个字符是不是数字、有没有括号、有没有分隔符……非常麻烦。

有了正则表达式之后：一行 `r"1[3-9]\d{9}"` 就能匹配手机号。

应用场景：

- 用户输入验证（手机号、邮箱、身份证号）
- 日志分析（提取时间、错误级别、消息内容）
- 爬虫数据清洗（从 HTML 中提取链接、文本）
- 文本替换（敏感词过滤、脱敏处理）

2. 正则表达式的基本语法（详细解释）

以下表格列出了最常用的元字符及其含义。注意：在 Python 中建议使用原始字符串 `r"..."`，这样反斜杠不会被转义。

元字符/语法	名称	说明	示例
.	点号	匹配除换行符外的任意一个字符	<code>a.c</code> 匹配 <code>abc</code> 、 <code>a7c</code> 、 <code>a</code> <code>c</code>
\d	数字	匹配一个数字 (0-9)	<code>\d{3}</code> 匹配三个连续数字
\D	非数字	匹配一个非数字字符	<code>\D+</code> 匹配连续的非数字
\w	单词字符	匹配字母、数字、下划线	<code>\w+</code> 匹配一个单词
\W	非单词字符	匹配非字母数字下划线	匹配标点符号、空格等
\s	空白字符	匹配空格、制表符、换行符等	<code>\s+</code> 匹配一段空白
\S	非空白字符	匹配非空白字符	
^	脱字符	匹配字符串的 开头	<code>^Hello</code> 匹配以 <code>Hello</code> 开头的字符串
\$	美元符	匹配字符串的 结尾	<code>world\$</code> 匹配以 <code>world</code> 结尾的字符串
*	星号	前一个字符出现 0 次或多次	<code>ab*</code> 匹配 <code>a</code> 、 <code>ab</code> 、 <code>abb</code> 、 <code>abbb...</code>
+	加号	前一个字符出现 1 次或多次	<code>ab+</code> 匹配 <code>ab</code> 、 <code>abb</code> ，不匹配 <code>a</code>
?	问号	前一个字符出现 0 次或 1 次	<code>ab?</code> 匹配 <code>a</code> 、 <code>ab</code>
{n}	花括号	前一个字符恰好出现 n 次	<code>\d{4}</code> 匹配四个数字 (如年份)
{n,m}	花括号	前一个字符出现 n 到 m 次	<code>[a-z]{2,5}</code> 匹配 2-5 个小写字母
[abc]	字符类	匹配中括号中的任意一个字符	<code>[aeiou]</code> 匹配任意一个元音字母
[^abc]	否定字符类	匹配除中括号内字符以外的任意字符	<code>[^0-9]</code> 匹配非数字字符
	竖线	或逻辑，匹配左边或右边的表达式	<code>cat dog</code> 匹配 <code>cat</code> 或 <code>dog</code>

元字符/语法	名称	说明	示例
<code>()</code>	括号	分组，可以提取子串，也可以应用量词	<code>(\d{4})-(\d{2})</code> 提取年、月
<code>(?:...)</code>	非捕获分组	分组但不捕获，不占用内存	<code>(?:com cn)</code> 只做匹配，不提取
<code>\b</code>	单词边界	匹配单词的开头或结尾	<code>\bword\b</code> 匹配独立的单词 word
<code>(?P<name>)</code>	命名分组	给分组起名字，方便提取	<code>(?P<year>\d{4})</code> 提取年份

① Note

特殊字符如 `. * + ? ^ $ | ( ) [ ] { } \` 在正则中都有特殊含义。如果要匹配这些字符本身，需要加上反斜杠转义，例如 `\.` 匹配点号，`\\` 匹配反斜杠。

3. Python 的 re 模块详解

语法标准写法：

Python中使用正则需要导入官方内置模块 `re`

```
python
1 # 固定开头
2 import re
3
4 # 最常用通用语法
5 re.函数名(正则规则, 目标文本)
```

拆解说明：

- `re`：Python内置正则模块（关键字，固定不变）
- 函数名：可变，常用：`match`、`search`、`findall`
- 正则规则：可变，我们自己定义的匹配公式（核心）
- 目标文本：可变，需要处理的原始字符串

3.1 re.match(pattern, string) —— 从字符串开头匹配

定义: `re.match` 尝试从字符串的起始位置匹配模式。如果开头不匹配, 直接返回 `None`。

```
python
1  import re
2
3  # 匹配成功
4  result = re.match(r"\d+", "123abc")
5  print(result)                    # <re.Match object;
    span=(0, 3), match='123'>
6  print(result.group())            # 123
7  print(result.span())             # (0, 3) 匹配的起始和结束索引
8
9  # 匹配失败 (开头不是数字)
10 result2 = re.match(r"\d+", "abc123")
11 print(result2)                   # None
```

常用方法:

- `group()`: 返回匹配到的字符串。
- `groups()`: 返回所有捕获组 (元组形式)。
- `start()`: 匹配的起始索引。
- `end()`: 匹配的结束索引。
- `span()`: 返回 `(start, end)` 元组。

3.2 re.search(pattern, string) —— 搜索第一次出现

定义: `re.search` 在整个字符串中搜索匹配模式的第一个位置。

```
python
```

```
1 result = re.search(r"\d+", "abc123def456")
2 print(result.group())           # 123 (只返回第一个匹配)
```

`match` 与 `search` 的区别:

- `match` 必须从开头匹配, `search` 可以在任何位置。
- 如果字符串开头就符合, 两者结果相同。

3.3 `re.findall(pattern, string)` —— 返回所有匹配的列表

定义: `re.findall` 返回字符串中所有非重叠匹配的结果, 以列表形式返回。

```
python
1 result = re.findall(r"\d+", "a1b22c333")
2 print(result)           # ['1', '22', '333']
3
4 # 如果有分组, 返回元组列表
5 result = re.findall(r"(\d+)-(\d+)", "12-34, 56-78")
6 print(result)           # [('12', '34'), ('56', '78')]
```

3.4 `re.finditer(pattern, string)` —— 返回所有匹配的迭代器

定义: `re.finditer` 返回一个迭代器, 每个元素是 `Match` 对象。适合处理大量匹配结果, 节省内存。

```
python
1 for match in re.finditer(r"\d+", "a1b22c333"):
2     print(match.group(), match.span())
3 # 1 (1,2)
4 # 22 (3,5)
5 # 333 (6,9)
```

3.5 re.sub(pattern, repl, string, count=0) —— 替换

定义: `re.sub` 将匹配到的内容替换为指定的字符串。

```
python
1 text = "我的电话是12345, 他的电话是67890"
2 new_text = re.sub(r"\d+", "***", text)
3 print(new_text)           # 我的电话是***, 他的电话是
                             ***
4
5 # 限制替换次数
6 new_text = re.sub(r"\d+", "***", text, count=1)
7 print(new_text)           # 我的电话是***, 他的电话是
                             67890
```

`repl` 可以是函数, 使替换更加灵活:

```
python
1 def mask_phone(match):
2     phone = match.group()
3     return phone[:3] + "****" + phone[7:]
4
5 text = "联系我:13812345678"
6 result = re.sub(r"1[3-9]\d{9}", mask_phone, text)
7 print(result)             # 联系我:138****5678
```

3.6 re.split(pattern, string, maxsplit=0) —— 按模式切分

定义: `re.split` 根据匹配到的模式切分字符串, 返回列表。

```
python
1 text = "apple, banana; orange|grape"
```

```

2 words = re.split(r"[;,|]+", text)
3 print(words) # ['apple', 'banana',
               'orange', 'grape']

```

3.7 re.compile(pattern) —— 编译正则表达式

当同一个正则表达式需要多次使用时，预编译可以提升性能。

```

python
1 phone_re = re.compile(r"1[3-9]\d{9}")
2 print(phone_re.search("我的号码13812345678").group()) #
  13812345678
3 print(phone_re.findall("张三13811111111,李四13922222222")) #
  ['13811111111', '13922222222']

```

4. 分组与提取（重要）

使用括号 `()` 可以标记要提取的部分。`group(0)` 是整个匹配，`group(1)` 是第一个括号内的内容，依此类推。

```

python
1 date = "2025-05-18"
2 match = re.search(r"(\d{4})-(\d{2})-(\d{2})", date)
3 if match:
4     print(match.group(0)) # 2025-05-18
5     print(match.group(1)) # 2025
6     print(match.group(2)) # 05
7     print(match.group(3)) # 18
8     year, month, day = match.groups() #
  ('2025', '05', '18')

```

使用命名分组，代码更清晰：

```

python

```



```

1 match = re.search(r"(?P<year>\d{4})-(?P<month>\d{2})-(?P<day>\d{2})", date)
2 print(match.group("year"))      # 2025
3 print(match.groupdict())        #
                                   {'year': '2025', 'month': '05', 'day': '18'}

```

5. 常见正则表达式案例

用途	正则表达式示例	说明
手机号（中国大陆）	<code>r"1(3 4 5 6 7 8 9)\d{9}"</code>	共11位，第二位是3-9
邮箱	<code>r"[a-zA-Z0-9_-]+@[a-zA-Z0-9_-]+(?:\.[a-zA-Z0-9_-]+)*"</code>	允许点号和横线
身份证（18位）	<code>r"[1-9]\d{5}(18 19 ([23]\d))\d{2}((0[1-9]) (10 11 12))([0-2][1-9]) 10 20 30 31)\d{3}[0-9Xx]"</code>	最后一位可能是数字或X/x
IPv4地址	<code>r"((25[0-5] 2[0-4]\d 1\d{2} [1-9]?\d)\.){3}(25[0-5] 2[0-4]\d 1\d{2} [1-9]?\d)"</code>	0.0.0.0 ~ 255.255.255.255
中文字符	<code>r"[\u4e00-\u9fa5]+"</code>	Unicode 中文字段范围

6. 贪婪匹配与非贪婪匹配

定义：默认情况下 `*` `+` `?` `{n,m}` 是**贪婪**的，会匹配尽可能多的字符。在后面加 `?` 变成**非贪婪**（懒惰）模式，匹配尽可能少的字符。

```

python
1 text = "<div>content</div><span>more</span>"
2
3 # 贪婪匹配（尽可能长）
4 greedy = re.search(r"<.*>", text)

```

```
5 print(greedy.group()) # <div>content</div>
   <span>more</span>
6
7 # 非贪婪匹配（遇到第一个 > 就停止）
8 lazy = re.search(r"<.*?>", text)
9 print(lazy.group()) # <div>
```

7. 课堂练习（正则）

- 编写正则表达式，验证用户输入的手机号是否合法（以1开头，第二位3-9，共11位数字）。
- 从文本 "姓名：张三，邮箱：zhangsan@example.com，电话：13812345678" 中提取出邮箱和电话。
- 使用 `re.sub` 将文本中所有的连续数字（如 123）替换为 [NUM]。
- 编写一个函数 `hide_phone(text)`，将文本中所有手机号的中间四位替换为 ****。

第十九章：浅拷贝和深拷贝

在 Python 编程中，**拷贝**（Copy）是指基于一个已有对象创建出一个新对象的操作。表面上看只是“复制一份”，但实际工作中会分为**浅拷贝**和**深拷贝**两种方式，它们对嵌套对象（例如列表中包含列表）的处理完全不同。

1. 为什么需要区分拷贝方式？

Python 中的变量并不直接存储值，而是存储**对象的引用**。当我们写出 `a = [1, 2, 3]` 时，`a` 实际上指向一块存放着列表数据的内存区域。如果执行 `b = a`，只是多了一个指向同一块内存的引用，并没有创建新列表。

```
python
1 a = [1, 2, 3]
2 b = a
```

```
3 | b.append(4)
4 | print(a) # [1, 2, 3, 4] , a 也跟着变了
```

为了避免这种“一改俱改”的情况，我们需要**显式地创建副本**，让新对象拥有自己的内存空间。而针对嵌套结构，副本的深度又分为浅拷贝和深拷贝。

2. 浅拷贝 (Shallow Copy)

浅拷贝创建一个新的复合对象（如新列表），然后将**原对象中的元素引用逐个复制到新对象中**。也就是说：

- 对于不可变元素（整数、字符串、元组等），复制引用和复制值效果一样（因为它们本身不可变）。
- 对于可变元素（如子列表、子字典），浅拷贝只复制引用，而**不会递归创建子对象的副本**。

因此，浅拷贝得到的对象中，内层的可变元素仍然与原始对象共享。

2.1 实现浅拷贝的方式

- 使用标准库 `copy` 模块的 `copy.copy()` 函数。
- 对于列表，可以使用 `list.copy()` 方法或切片 `[:]`。
- 对于字典，可以使用 `dict.copy()` 方法。

```
python
1 | import copy
2 |
3 | original = [1, 2, [3, 4]]
4 | shallow = copy.copy(original) # 或 original.copy()
```

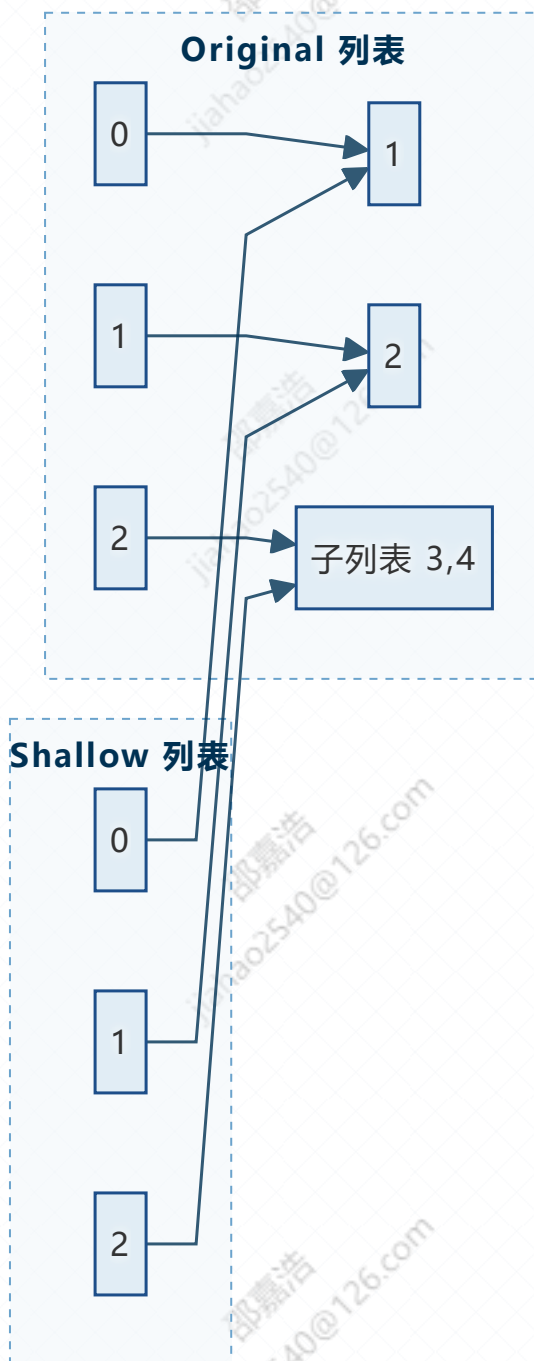
2.2 验证浅拷贝的“共享”特性

```
python
1 | original = [1, 2, [3, 4]]
2 | shallow = original.copy()
3 |
4 | shallow[0] = 100 # 修改第一层不可变元素
```

```
5 shallow[2].append(5)           # 修改嵌套的可变元素
6
7 print(original)                 # [1, 2, [3, 4, 5]]
8 print(shallow)                 # [100, 2, [3, 4, 5]]
```

- 修改 `shallow[0]`（整数，不可变）不会影响原列表。
- 修改 `shallow[2]`（子列表，可变）同时影响原列表，因为它们引用的是同一个内层列表对象。

2.3 浅拷贝的可视化



第一层的元素 1、2 是独立的（不可变元素共享也没事），但第二层的子列表是同一个对象。

3. 深拷贝 (Deep Copy)

深拷贝 创建一个全新的复合对象，并**递归地复制**原对象内部所有层级的子对象。最终产生一个与原对象完全独立、不共享任何可变元素的副本。

3.1 实现深拷贝

使用 `copy` 模块的 `deepcopy()` 函数。

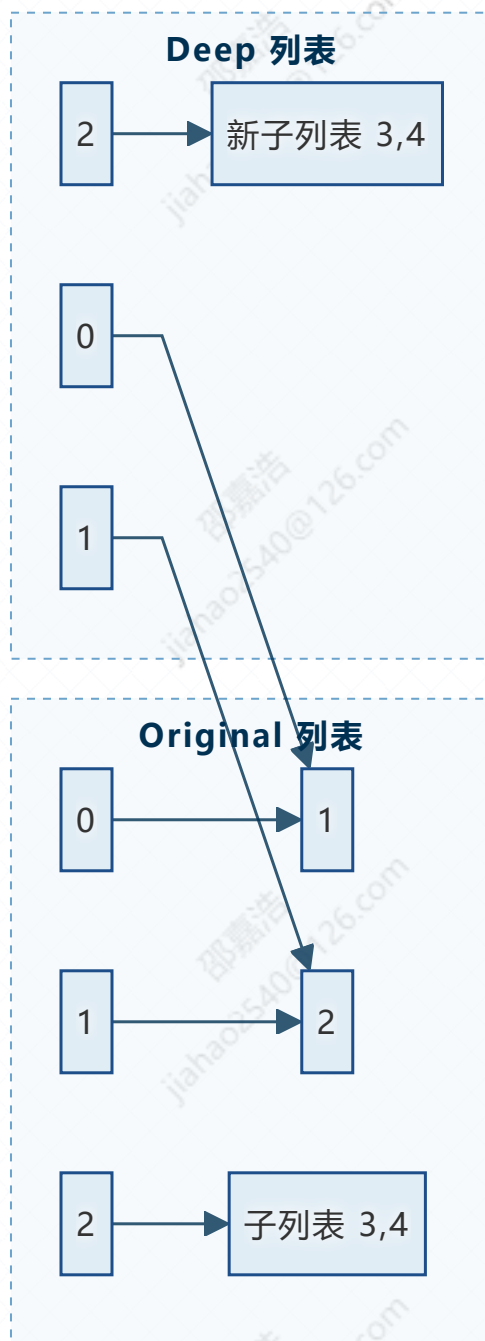
```
python
1 import copy
2
3 original = [1, 2, [3, 4]]
4 deep = copy.deepcopy(original)
```

3.2 验证深拷贝的独立性

```
python
1 original = [1, 2, [3, 4]]
2 deep = copy.deepcopy(original)
3
4 deep[2].append(5)
5
6 print(original)    # [1, 2, [3, 4]]    原对象没变
7 print(deep)       # [1, 2, [3, 4, 5]] 只影响副本
```

任何修改都限于深拷贝对象本身，不会影响原始对象。

3.3 深拷贝的可视化



内层子列表也被完全复制，两者彻底独立。

4. 浅拷贝与深拷贝的对比

特性	浅拷贝	深拷贝
顶层对象	新建	新建
内层可变对象	共享引用	递归复制新对象
修改内层元素	会影响原对象	不会影响原对象
执行速度	较快	较慢（特别是大对象）
适用场景	仅需一层独立，内部无需修改	需要完全独立的数据副本

5. 实际应用中的选择

- **浅拷贝适合**：当对象只有一层，或者内层元素不会被修改时。例如快速复制一份配置字典，只修改顶层键值。
- **深拷贝适合**：当对象嵌套复杂且需要与原对象彻底断开关系时。例如在算法中对复杂状态做回溯，或并行处理数据时避免数据污染。

6. 注意事项

- **不可变元素**：对于字符串、数字、元组等不可变元素，浅拷贝和深拷贝效果一致，因为它们本身不可修改，无须深拷贝。
- **循环引用**：`copy.deepcopy` 能够正确处理对象间的循环引用，避免无限递归。
- **性能**：深拷贝会递归遍历整个对象树，对于大型数据结构可能开销较大，需要根据实际需求权衡使用。

7. 总结

- **浅拷贝**：只复制对象本身，内层可变元素仍与原对象共享。
- **深拷贝**：递归复制整个对象树，生成完全独立的新对象。

第 二十 章：命名空间与作用域 —— 变量名字的查找规则

1. 为什么需要理解命名空间与作用域？

定义：

- **命名空间 (Namespace)**：是一个存储变量名与对象之间对应关系的“容器”。可以理解为一个字典，键是变量名，值是变量指向的对象。
- **作用域 (Scope)**：是程序中可以访问某个命名空间的区域。换句话说，作用域决定了在代码的某个位置，你能“看到”哪些变量。

大白话：

你在一个公司里，有不同级别的“地盘”：

- 你自己的工位（局部作用域）：只有你一个人放东西。
- 你所在的部门（外层作用域）：部门同事可以共享一些资料。
- 整个公司（全局作用域）：所有员工都能看到的公共区域。
- 社会通用的常识（内置作用域）：比如“太阳从东边升起”这种人人知道的东西。

Python 的变量查找就是按照 **从内到外** 的顺序，依次在这些“地盘”里找名字。

为什么重要：

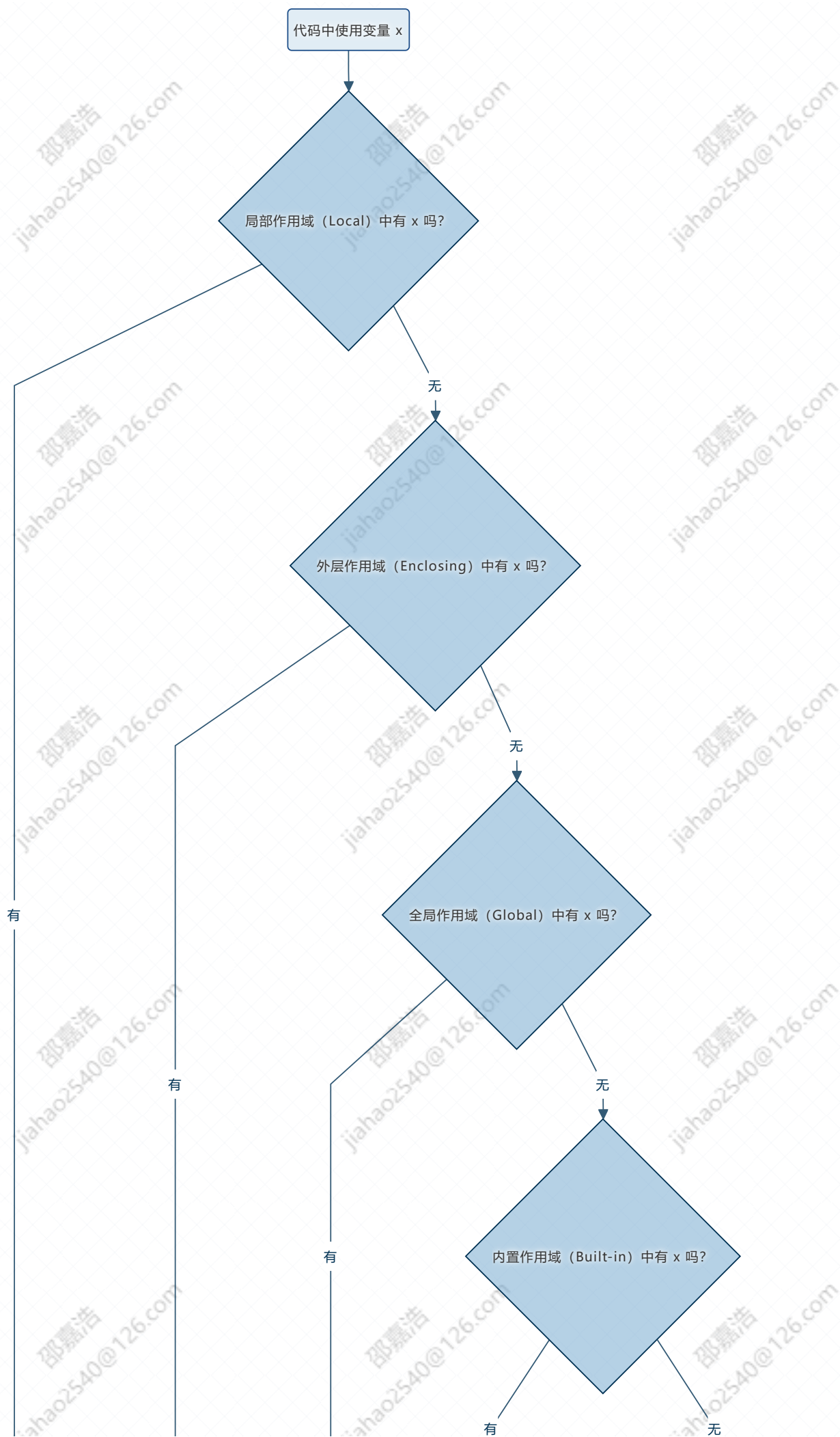
- 避免变量名冲突（不同函数可以有同名的局部变量，互不影响）。
- 理解为什么函数内部不能直接修改全局变量。
- 理解闭包中 `nonlocal` 的作用。
- 写出可预测、无副作用的代码。

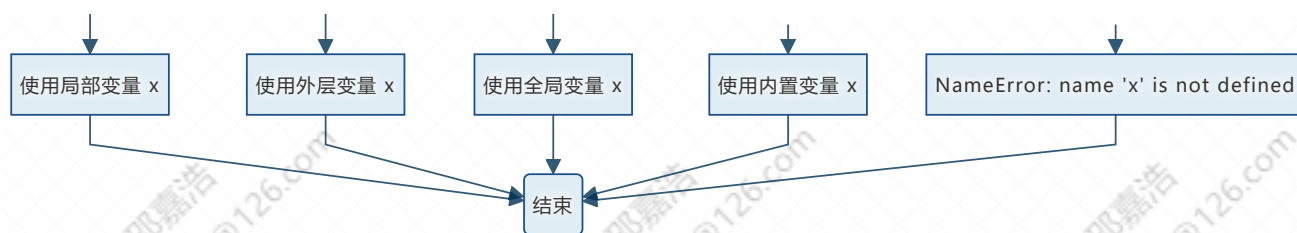
2. Python 的四种作用域（LEGB 规则）

Python 查找变量时，按照以下顺序搜索，一旦找到就停止，如果全部找不到就抛出 `NameError`。

缩写	英文名称	中文名称	说明
L	Local	局部作用域	函数内部通过赋值定义的变量，或者函数的参数。
E	Enclosing	外层作用域	在嵌套函数中，外层函数的作用域。常用于闭包。
G	Global	全局作用域	当前模块（.py 文件）顶层定义的变量。
B	Built-in	内置作用域	Python 解释器启动时预先加载的名称，如 <code>print</code> 、 <code>len</code> 、 <code>range</code> 、 <code>Exception</code> 等。

流程图：LEGB 查找顺序





3. 局部作用域 (Local)

定义：在函数内部使用赋值语句（`=`）创建的变量，或者函数的参数，都属于局部作用域。局部变量只能在函数内部访问，函数执行完毕后，局部变量会被销毁。

大白话：你办公桌上的私人笔记本，只有你自己能用，下班就锁进柜子。

示例 1：局部变量的定义与访问

```
python
1  def func():
2      x = 10    # 局部变量
3      print("函数内部:", x)
4
5  func()
6  # print(x)    # 如果取消注释，会报错 NameError: name 'x' is not defined
```

示例 2：局部变量与全局变量同名，互不影响

```
python
1  y = 100    # 全局变量
2
3  def test():
4      y = 200    # 局部变量，与全局变量同名
5      print("函数内部 y =", y)
6
7  test()
8  print("函数外部 y =", y)
```

4. 全局作用域 (Global)

定义：在函数外部、模块顶层定义的变量属于全局作用域。全局变量可以在整个模块的任何地方被读取，但不能直接在函数内部修改（除非使用 `global` 关键字）。

大白话：公司公告栏上的信息，所有人都能看到。但是如果你想修改公告栏上的内容，必须经过“审批”（用 `global` 声明）。

示例 1：在函数内部读取全局变量（不需要 `global`）

```
python
1  message = "Hello"    # 全局变量
2
3  def show():
4      print(message)    # 可以直接读取
5
6  show()                # 输出 Hello
```

- 全局变量 `message` 定义在顶层。
- 函数 `show` 内部没有定义同名的局部变量，所以 Python 按照 LEGB 规则找到了全局作用域中的 `message`，正常打印。

示例 2：尝试在函数内部修改全局变量（错误做法）

```
python
1  count = 0
2
3  def increment():
4      count += 1        # 试图修改全局变量
5      print(count)
6
7  increment()
8  # 报错: UnboundLocalError: local variable 'count' referenced
    before assignment
```

错误解释：

- 当函数内部有 `count += 1` 这样的赋值操作时，Python 会认为 `count` 是一个局部变量。

- 但在执行 `count += 1` 时，局部变量 `count` 还没有被赋值（没有初始值），所以报错。
- 这就是为什么不能直接修改全局变量。

示例 3：使用 `global` 正确修改全局变量

```
python
1  def increment():
2      global count    # 声明 count 是全局变量
3      count += 1
4      print(count)
5
6  increment()    # 输出 1
7  print(count)  # 输出 1（全局变量被修改了）
```

⚠ Caution

尽量少用 `global`，因为它会让函数产生“副作用”，即函数的行为依赖于外部状态，并且会改变外部状态。这会使代码难以调试和理解。优先考虑通过参数传递和返回值来交互。

5. 外层作用域（Enclosing）与 `nonlocal`

定义：当函数内部再定义函数（嵌套函数）时，内部函数可以访问外部函数的变量，这种外部函数的作用域称为“外层作用域”（Enclosing）。如果要修改外层函数的变量，需要使用 `nonlocal` 关键字。

大白话：你（内部函数）可以看部门主管（外部函数）的笔记本，但不能直接在上面写字；如果你非要改，需要先声明“我这不是在写自己的笔记本，而是在改主管的”。

示例 1：内部函数读取外层变量（不需要 `nonlocal`）

```
python
1  def outer():
2      x = 10    # 外层变量
3      def inner():
4          print(x)    # 可以读取 Python 在自己的局部作用域没找到
                        # 'x'，然后去外层作用域（'outer' 的作用域）
```

```
5     inner()
6
7 outer()    # 输出 10
```

示例 2：尝试修改外层变量（错误做法）

```
python
1  def outer():
2      x = 10
3      def inner():
4          x += 1    # 试图修改外层变量
5          print(x)
6      inner()
7
8  outer()
9  # 报错: UnboundLocalError: local variable 'x' referenced
    before assignment
```

错误解释：

- 和全局变量类似，当在 `inner` 中执行 `x += 1`，Python 默认把 `x` 当作 `inner` 的局部变量。
- 但局部 `x` 没有初始值，所以报错。

示例 3：使用 `nonlocal` 正确修改外层变量

```
python
1  def outer():
2      x = 10
3      def inner():
4          nonlocal x    # 告诉 Python，这个 x 不是局部变量，而是从
                          最近的外层函数（outer）中去找。
5          x += 1
6          print(x)
7      inner()
8      print("outer 中的 x:", x)
9
```

```
10 outer()
```

示例 4: `nonlocal` 与嵌套多层的区别

```
python
1 def outer():
2     x = "outer x"
3     def middle():
4         x = "middle x"    # 这一层的 x
5         def inner():
6             nonlocal x    # 这里 nonlocal 会找到最近的外层作用域
7             # 中的 x, 即 middle 中的 x
8             x = "inner changed"
9             print("inner:", x)
10            inner()
11            print("middle:", x)
12        middle()
13        print("outer:", x)
14    outer()
```

- `nonlocal` 不会一直找到最外层，而是找到最近的（第一个）外层作用域中的同名变量。
- 所以这里修改的是 `middle` 中的 `x`，而不是 `outer` 中的 `x`。

⚠ Caution

`nonlocal` 只能用于嵌套函数中，不能用于全局变量（全局变量用 `global`）。
`nonlocal` 会从当前函数的外层开始依次向外搜索，但不会搜索到全局作用域。

6. 内置作用域 (Built-in)

定义：Python 解释器启动时，会自动加载一些内置的函数、异常、常量，它们存放在内置作用域中。你可以在任何地方直接使用它们，无需导入。

常见内置名称：

- 函数: `print()`、`len()`、`range()`、`input()`、`type()`、`isinstance()` 等。
- 异常: `ValueError`、`TypeError`、`IndexError`、`KeyError` 等。
- 常量: `True`、`False`、`None`。

示例: 查看内置作用域

```
python
1 import builtins
2 print(dir(builtins)) # 列出所有内置名称
```

注意: 如果你在局部或全局作用域定义了一个与内置名称同名的变量, 就会“遮蔽”内置变量。

```
python
1 len = 100 # 全局变量 len 覆盖了内置函数 len
2 print(len) # 输出 100
3 # print(len([1,2,3])) # 报错, 因为 len 现在是整数, 不能当函数调用
```

⚠ Caution

避免使用内置名称作为变量名 (如 `list`、`dict`、`str`、`len`、`sum` 等), 以免造成混淆。

7. 命名空间的生命周期

命名空间类型	创建时机	销毁时机
内置命名空间	Python 解释器启动时	解释器退出
全局命名空间	模块被导入时 (或脚本直接运行时)	解释器退出, 或模块被显式删除
局部命名空间	函数被调用时	函数返回或抛出未捕获异常后
外层命名空间	外层函数被调用时	外层函数返回后 (除非闭包引用)

示例: 验证局部变量生命周期

```
1 def test():
2     local_var = "我是局部变量"
3     print(locals())    # {'local_var': '我是局部变量'}
4     return
5
6 test()
7 # print(local_var)    # 报错，局部变量已经销毁
```

8. 查看当前作用域的变量： `locals()` 和 `globals()`

- `locals()`：返回当前局部作用域中所有变量的字典。
- `globals()`：返回当前全局作用域中所有变量的字典。

```
1 a = 1
2 b = 2
3
4 def func():
5     c = 3
6     print("局部变量:", locals())
7     print("全局变量:", globals())
8
9 func()
```

输出（部分）：

```
1 局部变量: {'c': 3}
2 全局变量: {'a': 1, 'b': 2, 'func': <function func at ...>,
...}
```

第二十一章：迭代器（Iterator）—— 逐一访问的机制

1. 什么是迭代？

定义：迭代（Iteration）是指**重复地从一个对象中逐个取出元素**的过程。例如，用 `for` 循环遍历列表，就是典型的迭代。

生活类比：你有一个书架，你一本一本地把书拿出来看，看完一本再拿下一本，这就是迭代。你不需要知道书架里面书是怎么排列的，只要能一本本取出就行。

2. 可迭代对象（Iterable）与迭代器（Iterator）

2.1 可迭代对象（Iterable）

定义：实现了 `__iter__()` 方法的对象，或者实现了 `__getitem__()` 方法的对象，就是可迭代对象。常见的可迭代对象有：列表、元组、字符串、字典、集合、文件对象等。

判断方法：使用 `isinstance(obj, collections.abc.Iterable)`。

```
python
1  from collections.abc import Iterable
2
3  print(isinstance([1,2,3], Iterable))    # True
4  print(isinstance("hello", Iterable))    # True
5  print(isinstance(123, Iterable))        # False（整数不可迭代）
```

2.2 迭代器（Iterator）

定义：迭代器是一个**记住了遍历位置**的对象。它必须同时实现 `__iter__()` 和 `__next__()` 方法。

- `__iter__()` 返回迭代器自身。
- `__next__()` 返回下一个元素，如果没有元素了，抛出 `StopIteration` 异常。

判断方法：`isinstance(obj, collections.abc.Iterator)`。

```
python
1  from collections.abc import Iterator
2
3  lst = [1, 2, 3]
4  it = iter(lst)          # iter() 调用 lst.__iter__()
5  print(isinstance(it, Iterator))  # True
6  print(isinstance(lst, Iterator))  # False (列表本身不是迭代器)
```

2.3 迭代器的工作原理

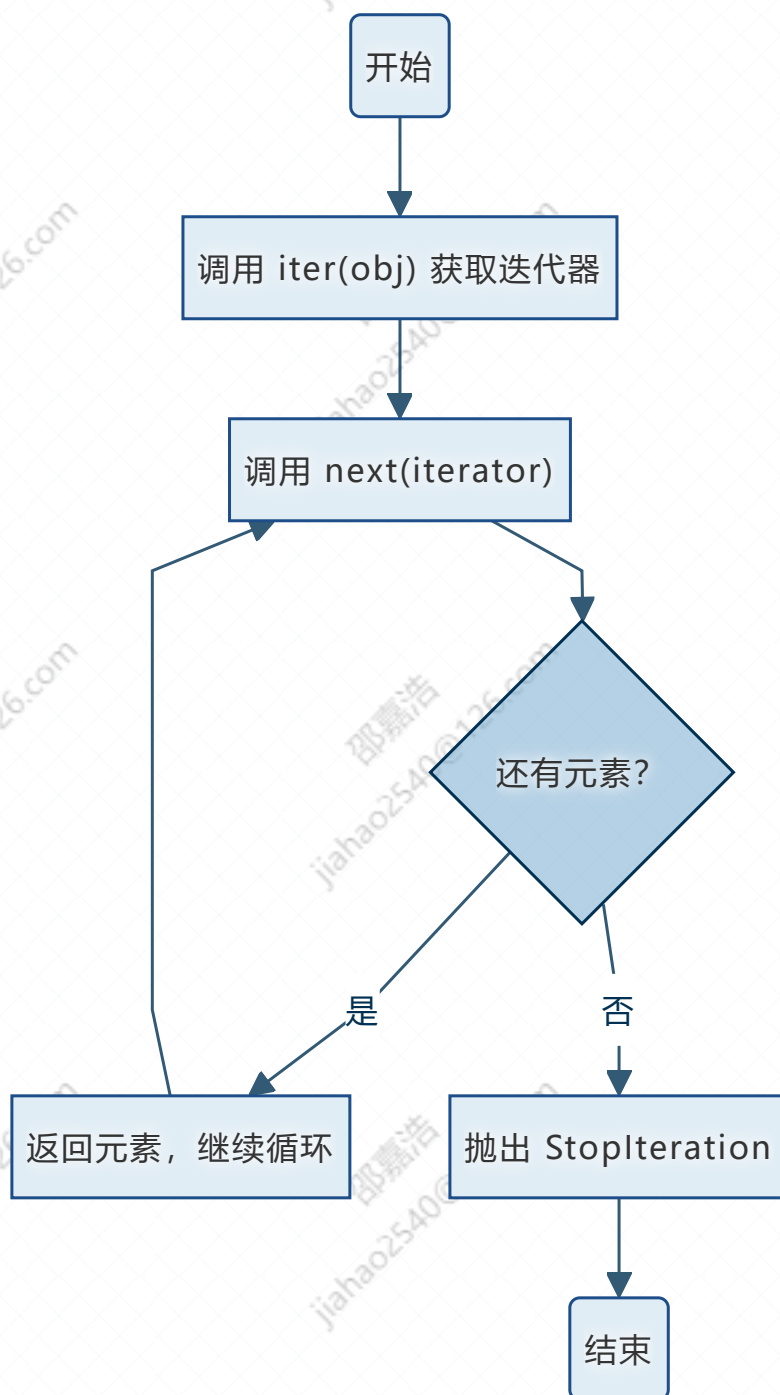
```
python
1  lst = [10, 20, 30]
2  it = iter(lst)
3
4  print(next(it))  # 10
5  print(next(it))  # 20
6  print(next(it))  # 30
7  print(next(it))  # 抛出 StopIteration 异常
```

for 循环的本质：

```
python
1  # 下面这个循环
2  for i in lst:
3      print(i)
4
5  # 等价于
6  it = iter(lst)
7  while True:
8      try:
```

```
9         i = next(it)
10        print(i)
11    except StopIteration:
12        break
```

流程图：迭代器的工作过程



3. 自定义迭代器

通过实现 `__iter__` 和 `__next__` 方法，我们可以创建自己的迭代器。

示例：实现一个从 `start` 到 `end-1` 的迭代器（类似 `range`）

```
python
1 class MyRange:
2     def __init__(self, start, end):
3         self.current = start
4         self.end = end
5
6     def __iter__(self):
7         return self # 迭代器返回自身
8
9     def __next__(self):
10        if self.current >= self.end:
11            raise StopIteration
12        value = self.current
13        self.current += 1
14        return value
15
16 # 使用
17 for i in MyRange(1, 5):
18     print(i) # 1 2 3 4
```

示例：迭代器可以无限生成数据

```
python
1 class Countdown:
2     def __init__(self, start):
3         self.n = start
4
5     def __iter__(self):
6         return self
7
8     def __next__(self):
```

```

9         if self.n < 0:
10             raise StopIteration
11         value = self.n
12         self.n -= 1
13         return value
14
15 cd = Countdown(3)
16 for num in cd:
17     print(num)    # 3 2 1 0

```

4. 迭代器的特点与限制

特点	说明
只能向前	不能回头获取前一个元素。
一次性	迭代完所有元素后，迭代器就失效了，不能再使用。
惰性计算	不会一次性把所有元素都准备好，而是在调用 <code>next</code> 时才生成下一个。
节省内存	对于大量数据，不需要提前存储在内存中。
不支持 <code>len()</code>	你不知道它还能迭代多少次。



python

```

1 it = MyRange(1, 3)
2 print(next(it))    # 1
3 print(next(it))    # 2
4 print(next(it))    # StopIteration, 之后不能再使用

```

5. 常见内置迭代器

函数/方法	返回的迭代器类型	说明
<code>iter(lst)</code>	<code>list_iterator</code>	列表迭代器
<code>iter(dict)</code>	<code>dict_keyiterator</code>	字典键的迭代器
<code>enumerate(lst)</code>	<code>enumerate</code> 对象	产生 (索引, 值) 对
<code>zip(lst1, lst2)</code>	<code>zip</code> 对象	聚合多个可迭代对象
<code>map(func, iterable)</code>	<code>map</code> 对象	对每个元素应用函数
<code>filter(func, iterable)</code>	<code>filter</code> 对象	筛选元素
<code>reversed(lst)</code>	<code>reversed</code> 对象	反向迭代

python

```
1  for i, char in enumerate("abc"):
2      print(i, char)    # 0 a, 1 b, 2 c
3
4  for a, b in zip([1,2], ['a','b']):
5      print(a, b)      # 1 a, 2 b
```

6. 课堂练习 (迭代器)

- 编写一个迭代器 `EvenNumbers`，接收一个 `limit`，迭代返回 0 到 `limit-1` 之间的所有偶数。
- 判断字符串 `"hello"`、列表 `[1,2]`、文件对象 `open("test.txt")` 哪些是可迭代对象？哪些是迭代器？
- 不使用 `for` 循环，手动模拟 `for i in [1,2,3]: print(i)` 的过程（用 `iter` 和 `next` 和 `try-except`）。

第二十二章：生成器（Generator）——更优雅的迭代器

1. 什么是生成器？

定义：生成器是一种**特殊的迭代器**，使用 `yield` 关键字而不是 `return`。生成器函数在执行时遇到 `yield` 会暂停，返回 `yield` 后面的值，并保留当前状态；下次调用 `next()` 时，从暂停处继续执行。

大白话：普通函数就像一架无人机，飞完整个航线就结束。生成器函数就像一架可以随时悬停、随时继续飞的无人机，你每次问它要一个结果，它就给你一个，然后停在那里等你下一次指令。

优点：

- **懒加载：**按需生成，不一次性占用大量内存。
- **可暂停、可恢复：**适合处理流式数据（如大文件、网络数据流）。
- **代码简洁：**比手动实现迭代器简单得多。

2. 生成器函数（yield）

2.1 基本使用

```
python
1  def count_up_to(n):
2      i = 0
3      while i < n:
4          yield i
5          i += 1
6
7  # 调用生成器函数返回一个生成器对象（不是直接执行函数体）
8  gen = count_up_to(3)
9  print(type(gen))      # <class 'generator'>
10
```

```
11 print(next(gen))      # 0
12 print(next(gen))      # 1
13 print(next(gen))      # 2
14 print(next(gen))      # StopIteration
15
16 # 生成器也是可迭代对象，可以用 for 循环
17 for num in count_up_to(5):
18     print(num)          # 0 1 2 3 4
```

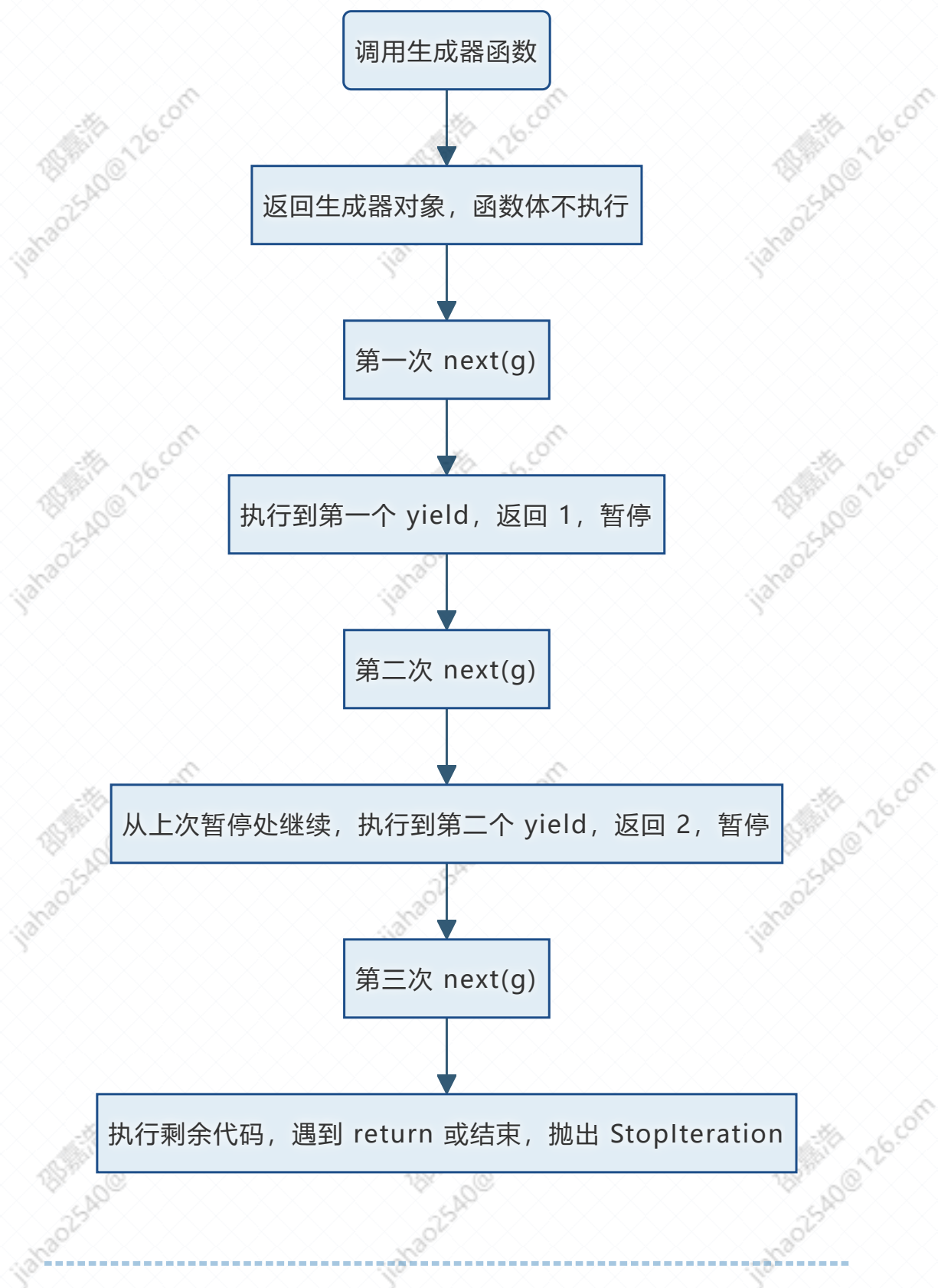
2.2 生成器的执行流程



The image shows a Python IDE window titled 'python'. The code editor contains the following Python code:

```
1 def simple_gen():
2     print("开始")
3     yield 1
4     print("继续")
5     yield 2
6     print("结束")
7
8 g = simple_gen()
9 print(next(g))      # 打印"开始", 然后返回1
10 print(next(g))     # 打印"继续", 然后返回2
11 print(next(g))     # 打印"结束", 然后抛出 StopIteration
```

流程图：生成器执行过程



3. 生成器的两种形式

3.1 生成器函数（使用 `yield`）

如上示例。

3.2 生成器表达式 (Generator Expression)

语法: (表达式 for 变量 in 可迭代对象 if 条件)

```
python
1 # 列表推导式 (立即生成整个列表)
2 squares_list = [x**2 for x in range(5)] # [0,1,4,9,16]
3
4 # 生成器表达式 (惰性计算)
5 squares_gen = (x**2 for x in range(5)) # <generator
  object>
6 print(next(squares_gen)) # 0
7 print(next(squares_gen)) # 1
8
9 # 可以用 sum 等函数直接操作生成器
10 total = sum(x**2 for x in range(5)) # 0+1+4+9+16=30
```

何时使用生成器表达式: 当数据量很大, 且只需要遍历一次时, 使用生成器表达式可以节省内存。如果需要多次遍历, 还是用列表推导式。

4. 生成器的应用实例

4.1 读取超大文件 (逐行处理)

假设有一个 10GB 的日志文件, 你不能一次性读入内存。

```
python
1 def read_large_file(file_path):
2     with open(file_path, 'r', encoding='utf-8') as f:
3         for line in f:
4             yield line.strip() # 每次返回一行
5
6 # 使用
7 for line in read_large_file("huge_log.txt"):
8     print(line) # 每次只处理一行, 内存中只有一行
```

4.2 无限序列生成器

定义：普通函数只能返回一个值（或一个容器），而生成器可以“无限”地产生值，因为它在每次 `yield` 后暂停，下次调用时继续。无限序列生成器通常配合 `next()` 和循环终止条件使用，不会一次性占用无限内存。

生活类比：想象一个永远会吐出数字的机器，你按一下按钮，它就吐出下一个数字。你不需要一次性拿走所有数字，机器也不会把所有数字提前算好。这就是无限序列生成器。

示例：斐波那契数列生成器

斐波那契数列的定义：从 0 和 1 开始，后面的每一项都是前两项之和。即 0, 1, 1, 2, 3, 5, 8, 13, 21, ...。它没有尽头（无限），但我们可以用生成器按需获取。

```
python
1  def fibonacci():
2      """无限生成斐波那契数列的生成器"""
3      a, b = 0, 1          # 初始化前两个数
4      while True:         # 无限循环，没有终止条件
5          yield a          # 每次返回当前的 a
6          a, b = b, a + b  # 更新：a 变成原来的 b, b 变成原来的
7                          a+b
8      # 使用示例：获取前 10 个斐波那契数
9      fib = fibonacci()
10     for i in range(10):
11         print(next(fib), end=" ") # 输出：0 1 1 2 3 5 8 13 21
34
```

4.3 流水线处理（生成器链）

定义：生成器链是指将多个生成器串联起来，每个生成器负责数据处理的一个环节，数据像流水线一样从一个生成器流向下一个。这种模式称为“生成器管道”（Generator Pipeline）。

生活类比：汽车制造流水线。第一个工位负责焊装车身，第二个工位负责喷漆，第三个工位负责安装发动机……每个工位只做一件事，但协同完成整辆车。生成器链也是类

似：第一个生成器读原始数据，第二个生成器清洗数据，第三个生成器计算结果。数据在每个环节被处理，然后传给下一个环节。

优点：

- **解耦**：每个环节独立，可以单独测试和复用。
- **内存友好**：数据以流的形式传递，不需要将整个数据集加载到内存。
- **灵活**：可以随意组合不同的生成器，形成新的流水线。

示例：从文件中读取数字 → 过滤偶数 → 平方

假设有一个文件 `numbers.txt`，每行一个整数，内容如下：

```
python
1 | 1
2 | 2
3 | 3
4 | 4
5 | 5
6 | 6
```

我们希望：读取文件 → 只保留偶数 → 计算偶数的平方 → 输出结果。

传统写法（一次性加载全部）：

```
python
1 | with open("numbers.txt") as f:
2 |     numbers = [int(line.strip()) for line in f] # 全部读入内存
3 | evens = [n for n in numbers if n % 2 == 0] # 再生成新列表
4 | squares = [n**2 for n in evens] # 再生成新列表
5 | print(squares) # [4, 16, 36]
```

问题：如果文件有几千万行，内存会爆掉。

生成器链写法（逐流处理）：

```
python
1 | def read_numbers(file_path):
```

```

2     """生成器1: 逐行读取文件, 产生整数"""
3     with open(file_path, 'r', encoding='utf-8') as f:
4         for line in f:
5             yield int(line.strip())
6
7     def filter_even(numbers):
8         """生成器2: 接收一个生成器 (或可迭代对象), 只产生偶数"""
9         for n in numbers:
10             if n % 2 == 0:
11                 yield n
12
13     def square(numbers):
14         """生成器3: 接收一个生成器, 产生每个数的平方"""
15         for n in numbers:
16             yield n ** 2
17
18     # 构建流水线
19     source = read_numbers("numbers.txt")      # 源头
20     evens = filter_even(source)               # 第二个环节
21     squares = square(evens)                  # 第三个环节
22
23     # 输出结果
24     for val in squares:
25         print(val)      # 4, 16, 36

```

更简洁的写法: 使用生成器表达式

```

1     with open("numbers.txt") as f:
2         numbers = (int(line.strip()) for line in f)
3         # 生成器表达式
4         evens = (n for n in numbers if n % 2 == 0)
5         # 生成器表达式
6         squares = (n ** 2 for n in evens)
7         # 生成器表达式

```



```
6 for val in squares:
7     print(val)
```

5. 生成器的其他特性

5.1 `send()` 方法向生成器发送值

定义：`send()` 是生成器对象的一个方法，它不仅可以恢复生成器的执行（像 `next()` 一样），还可以向生成器内部发送一个值，这个值会成为生成器内部 `yield` 表达式的结果。

大白话：

- `next(g)` 是“继续运行，我不给你东西”。
- `g.send(value)` 是“继续运行，并且给你一个礼物（value）”。
- 生成器内部遇到 `yield` 时，`yield` 本身可以接收外部发来的值，并存到一个变量中。

用途：

- 实现协程（Coroutine），让生成器既可以产出数据，也可以接收数据。
- 实现双向通信，比如生成器作为状态机，外部可以向它发送事件。

● 基本语法与示例

```
python
1 def accumulator():
2     """累加器生成器：接收数值，输出当前累加和"""
3     total = 0
4     while True:
5         value = yield total # yield total 返回当前累加和，并
                             等待接收外部发送的值
6         if value is None:
7             break
```

```

8         total += value
9
10    acc = accumulator()
11
12    # 第一次调用必须先启动生成器到第一个 yield
13    print(next(acc))          # 输出 0 (total 初始值)
14
15    print(acc.send(10))       # 发送 10, total 变成 10, 输出 10
16    print(acc.send(20))       # 发送 20, total 变成 30, 输出 30
17    print(acc.send(5))        # 发送 5, total 变成 35, 输出 35
18
19    acc.close()               # 关闭生成器

```

注意：`send()` 使用前必须先调用一次 `next()` 或 `send(None)` 让生成器运行到第一个 `yield`。

5.2 `close()` 关闭生成器

定义：`close()` 是生成器对象的一个方法，用于**手动关闭生成器**。关闭后，再调用 `next()`、`send()` 或 `throw()` 都会抛出 `StopIteration` 异常（正常结束的信号）。

大白话：

- 生成器就像一台机器，你不需要它了，按一下 `close()` 按钮，它会执行一些清理工作（比如释放资源），然后彻底停机。
- 生成器内部可以通过捕获 `GeneratorExit` 异常来执行清理代码，但不能产出任何值。

用途：

- 提前终止生成器，释放相关资源（如打开的文件、网络连接）。
- 通知生成器不再需要它，让生成器有机会执行收尾工作。

● 基本语法与示例



python

```

1  def my_gen():
2      try:
3          yield 1
4          yield 2
5          yield 3
6      except GeneratorExit:
7          print("生成器被关闭，执行清理")
8      finally:
9          print("无论如何都会执行（如关闭文件）")
10
11 g = my_gen()
12 print(next(g))    # 1
13
14 # 调用 close() 关闭生成器
15 g.close()         # 输出：生成器被关闭，执行清理\n 无论如何都会执行
16
17 # 关闭后不能再使用
18 # print(next(g))  # StopIteration

```

6. 生成器与迭代器的对比

特性	迭代器（手动实现）	生成器（yield）
实现方式	定义类，实现 <code>__iter__</code> 和 <code>__next__</code>	定义函数，使用 <code>yield</code>
代码量	较多	很少
状态保存	需要手动维护实例变量	自动保存局部变量和执行状态
可读性	一般	高
适用场景	需要复杂状态管理时	大多数简单迭代需求

7. 课堂练习（生成器）

1. 编写生成器函数 `my_range(start, end, step=1)`，模拟内置 `range` 的功能，不用支持负数步长，只做正数递增就行。
2. 使用生成器表达式计算 1 到 100 中所有奇数的平方和，要求不产生中间列表。
3. 生成器链：从一个包含整数的文件中读取数字，过滤出正数，再求出它们的平方，最后输出。每个环节用生成器实现。

第二十三章：装饰器（Decorator）——在不修改原函数的情况下增加功能

1. 为什么需要装饰器？

定义：装饰器（Decorator）是一种设计模式，它允许你**在不修改原函数代码**的前提下，给函数添加新的功能。

生活类比：

- 你有一个普通的煎饼（原函数）。
- 你想加个鸡蛋、加根火腿（增加功能）。
- 你不需要改变煎饼本身的配方，只需要在煎饼上面加料。
- 装饰器就像是“加料器”：它把原函数包起来，在外面添加新行为，然后返回一个增强版的函数。

常见应用场景：

- **日志记录：**自动打印函数调用时间、参数、返回值。
- **执行时间统计：**测试函数性能。
- **权限校验：**检查用户是否登录、是否有权限。
- **缓存：**将计算结果保存起来，避免重复计算。
- **重试机制：**当函数抛出异常时自动重试。
- **输入验证：**检查参数是否合法。

2. 函数作为参数和返回值（回顾）

装饰器的核心基础是：Python 中的函数是一等公民（First-class Citizen），意味着函数可以像普通变量一样：

- 赋值给其他变量
- 作为参数传递给另一个函数
- 作为另一个函数的返回值

示例 1：函数赋值给变量

```
python
1  def greet(name):
2      return f"Hello, {name}"
3
4  say_hello = greet    # 将函数赋值给变量
5  print(say_hello("小明"))    # Hello, 小明
```

代码解释：

- `greet` 是一个函数对象。
- `say_hello = greet` 没有调用函数，只是将函数对象的引用赋值给 `say_hello`。
- `say_hello("小明")` 等价于调用原来的 `greet` 函数。

示例 2：函数作为参数

```
python
1  def shout(func):
2      def wrapper(name):
3          original = func(name)
4          return original.upper() + "!"
5      return wrapper
6
7  def greet(name):
8      return f"Hello, {name}"
9
```



```
10 shouted_greet = shout(greet) # shout 接收 greet 作为参数
11 print(shouted_greet("小明")) # HELLO, 小明!
```

代码逐行解释：

- `shout` 函数接收一个参数 `func`，它期望 `func` 是一个函数。
- 在 `shout` 内部定义了一个嵌套函数 `wrapper`，它接收 `name`。
- `wrapper` 内部先调用 `func(name)` 得到原始结果，然后用 `upper()` 转成大写，再加感叹号。
- `shout` 返回 `wrapper` 函数（没有调用）。
- `shouted_greet = shout(greet)` 执行后，`shouted_greet` 就是 `wrapper` 函数。
- 调用 `shouted_greet("小明")` 实际上就是执行 `wrapper("小明")`。

这个模式就是装饰器的雏形：一个函数接收另一个函数，并返回一个新函数。

3. 基本装饰器语法（@ 语法糖）

定义：Python 提供了 `@` 符号来简化装饰器的使用。`@decorator` 写在函数定义之前，等价于 `func = decorator(func)`。

示例 1：最简单的装饰器（没有参数）

```
python
1 def logger(func):
2     def wrapper(*args, **kwargs):
3         print(f"调用函数: {func.__name__}")
4         result = func(*args, **kwargs)
5         print(f"返回值: {result}")
6         return result
7     return wrapper
8
9 @logger
10 def add(a, b):
11     return a + b
```

12

13 `add(3, 5)`

代码逐行解释：

- `logger` 是一个装饰器函数，它接收 `func`。
- `wrapper` 是内部函数，它使用 `*args, **kwargs` 来接收任意数量的参数。
- `wrapper` 先打印“调用函数”，然后调用原函数 `func`，打印返回值，最后返回结果。
- `@logger` 等价于 `add = logger(add)`。也就是说，原来的 `add` 函数被替换成了 `wrapper` 函数。
- 当我们调用 `add(3, 5)` 时，实际上调用的是 `wrapper(3, 5)`，然后 `wrapper` 内部再调用原始的 `add(3, 5)`。



`*args` 收集所有位置参数为元组，`**kwargs` 收集所有关键字参数为字典。这样装饰器可以适用于任何函数。

4. 保留原函数元信息 (`functools.wraps`)

问题：装饰器会覆盖原函数的 `__name__`、`__doc__`、`__module__` 等属性。这会导致调试困难，例如打印函数名会显示 `wrapper` 而不是原函数名。

示例：没有 `wraps` 的问题

```
1 def logger(func):
2     def wrapper(*args, **kwargs):
3         """wrapper 的文档"""
4         result = func(*args, **kwargs)
5         return result
6     return wrapper
7
8 @logger
9 def add(a, b):
```

python

```

10     """返回两数之和"""
11     return a + b
12
13 print(add.__name__)    # 输出 wrapper, 不是 add
14 print(add.__doc__)     # 输出 wrapper 的文档, 不是 "返回两数之和"

```

解决方案：使用 `functools.wraps(func)` 装饰器。它会将原函数的元信息复制到 `wrapper` 函数中。

```

python
1  import functools
2
3  def logger(func):
4      @functools.wraps(func)    # 这一行是关键
5      def wrapper(*args, **kwargs):
6          """wrapper 的文档（会被覆盖）"""
7          result = func(*args, **kwargs)
8          return result
9      return wrapper
10
11 @logger
12 def add(a, b):
13     """返回两数之和"""
14     return a + b
15
16 print(add.__name__)    # 输出 add
17 print(add.__doc__)     # 输出 返回两数之和

```



编写装饰器时，应该始终在内部函数上使用 `@functools.wraps(func)`，这是一个好习惯。

5. 带参数的装饰器

定义：有时装饰器本身也需要参数，例如指定重试次数、日志级别、缓存大小等。这需要三层嵌套函数。

结构：

```
python
1  def decorator_with_args(arg1, arg2):
2      def real_decorator(func):
3          def wrapper(*args, **kwargs):
4              # 使用 arg1, arg2
5              return func(*args, **kwargs)
6          return wrapper
7      return real_decorator
```

使用：

```
python
1  @decorator_with_args(10, "hello")
2  def my_func():
3      pass
```

示例：重试装饰器

```
python
1  import functools
2  import time
3
4  def retry(max_retries=3, delay=1):
5      """装饰器：函数失败时自动重试"""
6      def decorator(func):
7          @functools.wraps(func)
8          def wrapper(*args, **kwargs):
9              for attempt in range(max_retries):
10                 try:
11                     return func(*args, **kwargs)
12                 except Exception as e:
```

```

13         print(f"第 {attempt+1} 次尝试失败: {e}")
14         if attempt == max_retries - 1:
15             raise # 最后一次失败，抛出异常
16         time.sleep(delay)
17     return wrapper
18     return decorator
19
20 @retry(max_retries=3, delay=0.5)
21 def unstable_function():
22     import random
23     if random.random() < 0.7:
24         raise ValueError("随机失败")
25     return "成功"
26
27 print(unstable_function())

```

代码逐行解释：

- `retry` 是一个装饰器工厂，它接收参数 `max_retries` 和 `delay`，返回一个真正的装饰器 `decorator`。
- `decorator` 接收函数 `func`，返回 `wrapper`。
- `wrapper` 内部尝试 `max_retries` 次调用 `func`，每次失败后等待 `delay` 秒。
- 如果全部失败，最后将异常抛出。
- 调用 `@retry(3, 0.5)` 时，首先执行 `retry(3, 0.5)` 得到 `decorator`，然后 `@decorator` 装饰 `unstable_function`。

6. 多个装饰器叠加

定义：一个函数可以有多个装饰器，执行顺序是从下往上包裹，从上往下执行。

示例：

```

1 def deco1(func):

```

python


```

2     def wrapper():
3         print("deco1 before")
4         func()
5         print("deco1 after")
6     return wrapper
7
8     def deco2(func):
9         def wrapper():
10             print("deco2 before")
11             func()
12             print("deco2 after")
13         return wrapper
14
15     @deco1
16     @deco2
17     def hello():
18         print("hello")
19
20     hello()
21     """
22     输出
23         deco1 before
24         deco2 before
25         hello
26         deco2 after
27         deco1 after
28     """

```

详细过程：

1. `@deco2` 先执行：`hello = deco2(hello)`，此时 `hello` 变成了 `deco2` 中的 `wrapper`。
2. `@deco1` 接着执行：`hello = deco1(已经被deco2包装的hello)`，即 `deco1` 接收的参数是 `deco2` 返回的 `wrapper`。
3. 最终 `hello` 是 `deco1` 的 `wrapper`。

- 调用 `hello()` 时, 进入 `deco1` 的 `wrapper`, 打印 "deco1 before", 然后调用内部的 `func()`, 这个 `func` 就是 `deco2` 的 `wrapper`。
- 进入 `deco2` 的 `wrapper`, 打印 "deco2 before", 调用它内部的 `func()`, 这个 `func` 就是原始 `hello` 函数。
- 原始 `hello` 执行, 打印 "hello"。
- 返回到 `deco2` 的 `wrapper`, 打印 "deco2 after"。
- 返回到 `deco1` 的 `wrapper`, 打印 "deco1 after"。

7. 典型装饰器应用案例 —— 计时器装饰器

```
python
1  import time
2  import functools
3
4  def timer(func):
5      """计算函数执行时间"""
6      @functools.wraps(func)
7      def wrapper(*args, **kwargs):
8          start = time.perf_counter() # 高精度计时
9          result = func(*args, **kwargs)
10         end = time.perf_counter()
11         print(f"{func.__name__} 耗时 {end - start:.6f} 秒")
12         return result
13     return wrapper
14
15 @timer
16 def slow_function():
17     time.sleep(1)
18     return "done"
19
20 slow_function() # 输出: slow_function 耗时 1.000123 秒
```

8. 类装饰器

定义：除了用函数做装饰器，也可以用类做装饰器。类需要实现 `__init__` 和 `__call__` 方法。

示例：

```
python

1  class CountCalls:
2      def __init__(self, func):
3          self.func = func
4          self.count = 0
5
6      def __call__(self, *args, **kwargs):
7          self.count += 1
8          print(f"调用次数: {self.count}")
9          return self.func(*args, **kwargs)
10
11  @CountCalls
12  def say_hello():
13      print("Hello")
14
15  say_hello()
16  say_hello()
17  # 输出:
18  # 调用次数: 1
19  # Hello
20  # 调用次数: 2
21  # Hello
```

- `CountCalls` 类接收 `func` 作为初始化参数，并保存到 `self.func`。
- `__call__` 方法使得实例可以像函数一样被调用。
- `@CountCalls` 等价于 `say_hello = CountCalls(say_hello)`。
- 调用 `say_hello()` 时，实际上调用的是 `CountCalls` 实例的 `__call__` 方法。

总结与复习

概念	核心要点
正则表达式	<code>\d</code> 、 <code>\w</code> 、 <code>+</code> 、 <code>*</code> 、 <code>{n}</code> ； <code>re.match</code> （开头）、 <code>re.search</code> （搜索）、 <code>re.findall</code> （全找）、 <code>re.sub</code> （替换）；分组 <code>()</code> 提取信息；贪婪与非贪婪 <code>.*</code> 。
命名空间与作用域	LEGB 规则：局部(L) → 外层(E) → 全局(G) → 内置(B)； <code>global</code> 修改全局变量； <code>nonlocal</code> 修改外层变量； <code>locals()</code> / <code>globals()</code> 查看变量。
迭代器	实现 <code>__iter__</code> 和 <code>__next__</code> ； <code>iter()</code> 获取迭代器， <code>next()</code> 取值； <code>for</code> 循环本质；只能向前、一次性、惰性计算。
生成器	<code>yield</code> 函数；生成器表达式 <code>(x for x in ...)</code> ；无限序列、流水线处理； <code>send()</code> 发送值， <code>throw()</code> 抛异常， <code>close()</code> 关闭。
装饰器	函数作为参数和返回值； <code>@</code> 语法糖； <code>functools.wraps</code> 保留元信息；带参数装饰器（三层嵌套）；多个装饰器从下往上包裹，从上往下执行；常见应用：计时、缓存、权限、重试、日志。

课后作业

- 正则表达式综合**：编写一个函数 `extract_info(text)`，从文本中提取所有手机号、邮箱地址、URL，返回字典。测试文本：`"联系我: 13812345678 或 email@example.com 访问 https://python.org"`。
- 迭代器实现斐波那契**：编写一个迭代器 `Fibonacci`，可以指定最大项数，支持 `for` 循环和 `len()`（提示：需要额外记录生成的数量）。
- 生成器实现素数**：编写生成器 `primes()`，无限生成质数（从2开始）。使用埃拉托斯特尼筛法思路。然后找出第100个质数。

4. **装饰器综合**：实现三个装饰器，然后叠加在一个函数上：

- **@log**：打印调用信息。
- **@timer**：打印执行时间。
- **@retry(2)**：失败时重试2次。

测试函数 **risky_operation** 随机抛出异常，观察装饰器叠加效果。